

个人作业报告

题目：**大数据原理与技术**

作业二：知识图谱构建与可视化

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	王昌栋

2026 年 3 月 29 日

【摘　要】

在学习完大数据原理相关课程后[1]，完成本次作业（作业-2）要求构建一个小型领域知识图谱并提交简要报告与代码。具体要求如下：

任务要求：

- 自行选择一个小范围的主题或领域数据集（例如新闻片段、科研摘要、课程材料、产品描述等），并在报告中说明数据来源与采集方式；
- 使用 NLP 工具（如 spaCy）对文本数据进行预处理与信息抽取，完成实体抽取与关系抽取的流程；
- 使用 Neo4j 构建知识图谱，将抽取得到的实体与关系导入图数据库，并通过可视化展示关键节点与子图；

处理流程：

1) 数据准备：使用爬虫采数据集并清洗文本数据；
2) 实体与关系抽取：使用 spaCy 的命名实体识别（NER）识别实体类别，结合规则或简单模型抽取实体间的关系；
3) 图数据库构建：设计实体与关系的模式（labels 与 relationship types），使用 Cypher 或官方驱动将节点与边导入 Neo4j；
4) 可视化与分析：在 Neo4j Browser 或 Neo4j Bloom 中展示关键节点，筛选并截图作为报告结果；

提交说明：

- 提交包内容：代码、报告与必要的数据（或数据下载说明）；
- 文件命名格式：学号 + 姓名 + 作业 2.zip；
- 提交邮箱：BDSysu2026Spring@163.com；
- 截止时间：布置作业之后的下一个周日 24:00；
- 抄袭处理：严格禁止抄袭，发现抄袭将按零分处理。

关键词： 知识图谱，实体抽取，关系抽取，Neo4j，可视化

目录

1 实验细节 1
1.1 实验目标 1
1.2 方法实现 1
1.3 数据统计 1
1.4 实验内容 2
1.5 实验结果与分析 3
1.6 核心代码及说明 3
1.7 算法处理介绍 4
1.8 结果展示 5

2 总结 7
2.1 实验总结 7
2.2 主要成果 7

参考文献 8

附录 A 实验代码 9
A.1 数据预处理代码 9
A.2 关系提取代码 10
A.3 Neo4j 图谱构建代码 12

1 实验细节

1.1 实验目标

本次实验的目标是构建一个小型领域知识图谱，提取中山大学教师信息中的实体和关系，并将其导入到 Neo4j 数据库中进行可视化和分析。

1.2 方法实现

1.2.1 数据来源与预处理

- 数据来源于中山大学官网的教师个人主页。
- 使用 Python 脚本对原始数据进行清洗和预处理，生成结构化的 CSV 文件。

1.2.2 NLP 工具的使用

- 使用 spaCy 工具进行分词、命名实体识别（NER）和关系抽取。
- 提取的实体包括教师姓名、工作单位（学院）和所属大学。
- 提取的关系包括“工作于”和“隶属于”。

1.2.3 Neo4j 图谱构建

- 使用 py2neo 库连接 Neo4j 数据库。
- 将提取的实体和关系以 JSON 格式存储，并通过脚本将其导入 Neo4j 数据库。
- 在 Neo4j 中创建节点和边，节点包括 Person、Organization 和 University，边包括 WORKS_AT 和 BELONGS_TO。

1.3 数据统计

1.3.1 原始数据统计

原始数据包含中山大学中国语言文学系的教师信息，数据来源于官网教师个人主页。以下是原始数据的统计信息：

- 教师总人数：12608
- 涉及的学院：88

- 所属大学：1 所（中山大学）

1.3.2 知识图谱统计

基于原始数据构建的知识图谱包含以下点和边的数量：

- 节点总数：12608 个 Person 节点，88 个 Organization 节点，1 个 University 节点，总计 12697 个节点。
- 边总数：12608 条 WORKS_AT 关系，88 条 BELONGS_TO 关系，总计 12796 条边。
  - WORKS_AT 关系：12608 条（每个教师与其学院的关系）。
  - BELONGS_TO 关系：88 条（学院与大学的关系）。

1.3.3 统计说明

相关数据保存在 data 文件夹中，包含爬虫原数据、清洗后的数据以及导入 neo4j 的 JSON 文件。

1.4 实验内容

1.4.1 原始文本片段

```
**丁文俊**

中国语言文学系，文艺学教研室 副教授

丁文俊，男，广东东莞人，1988年生，文学博士，中山大学中国语言文学系副教授，硕士生导师。在中山大学中国语言文学系获得文学学士学位（2012.6）；在华东师范大学中国语言文学系文艺学专业师从朱国华教授，先后获得文学硕士学位（2015.6）和文学博士学位（2019.6）；2016年10月—2018年7月，获国家留学基金委资助，前往法国国立东方语言文化学院（ Institut National des Langues et Civilisations Orientales )访学；2019年9月—2025年4月，在中山大学中国语言文学系师从王坤教授，从事科研博士后和专职科研岗工作。

主要研究方向为文艺社会学，研究领域涵盖西方马克思主义美学、文化研究和晚明美学。

一、代表性研究成果
[西方文论方向]
```

```
[1] 丁文俊：《阿多诺美学的自然记忆与“感性.....

二、科研项目
[1] 2023年国家社科基金后期资助项目“记忆的政治：‘阿本之争’的历史境.....

三、联系方式
邮箱：dingwj7@mail.sysu.edu.cn
```

1.4.2 提取的实体与关系

- 实体：
  - Person: 丁文俊
  - Organization: 中国语言文学系
  - University: 中山大学
- 关系：
  - 丁文俊 WORKS_AT 中国语言文学系
  - 中国语言文学系 BELONGS_TO 中山大学

1.5 实验结果与分析

1.5.1 图谱覆盖范围

图谱包含了中山大学中国语言文学系的所有教师信息及其与学院、大学的关系。

1.5.2 准确性分析

实体和关系提取的准确率较高，但对于部分复杂描述可能存在漏提或误提的情况。

1.6 核心代码及说明

以下是实验中使用的部分核心代码：

```python
def extract_information(description):
    nlp = spacy.load("zh_core_web_sm")
    doc = nlp(description)
```

```
    # Extract name
    name_match = re.search(r"\*\*(.*?)\*\*", description)
    name = name_match.group(1) if name_match else ""

    # Extract workplace and department
    workplace = ""
    department = ""
    for ent in doc.ents:
        if ent.label_ == "ORG":
            if "大学" in ent.text:
                workplace = ent.text
            else:
                department = ent.text

    return {
        "name": name,
        "workplace": workplace,
        "department": department
    }
```

上述代码实现了从教师信息描述中提取核心实体（姓名、工作单位、部门）的功能：

- 使用 spaCy 加载中文语言模型进行分词和命名实体识别。
- 使用正则表达式提取教师姓名。
- 遍历命名实体，识别出工作单位和部门。

1.7 算法处理介绍

本实验的算法处理流程如下：

1) ** 数据预处理 **：对原始文本数据进行清洗，去除无关信息，确保数据格式统一。
2) ** 实体提取 **：使用 spaCy 进行命名实体识别，提取教师姓名、工作单位和所属大学。
3) ** 关系抽取 **：基于规则和上下文，提取教师与工作单位、工作单位与大学之间的关系。

4) ** 数据存储 **：将提取的实体和关系以 JSON 格式存储，便于后续导入 Neo4j。

5) ** 图谱构建 **：使用 py2neo 库将 JSON 数据导入 Neo4j，创建节点和边，构建知识图谱。

1.8 结果展示

1.8.1 关系图展示

由于图谱较大，对于硬件要求较高，以下展示部分关键节点和关系的可视化结果：

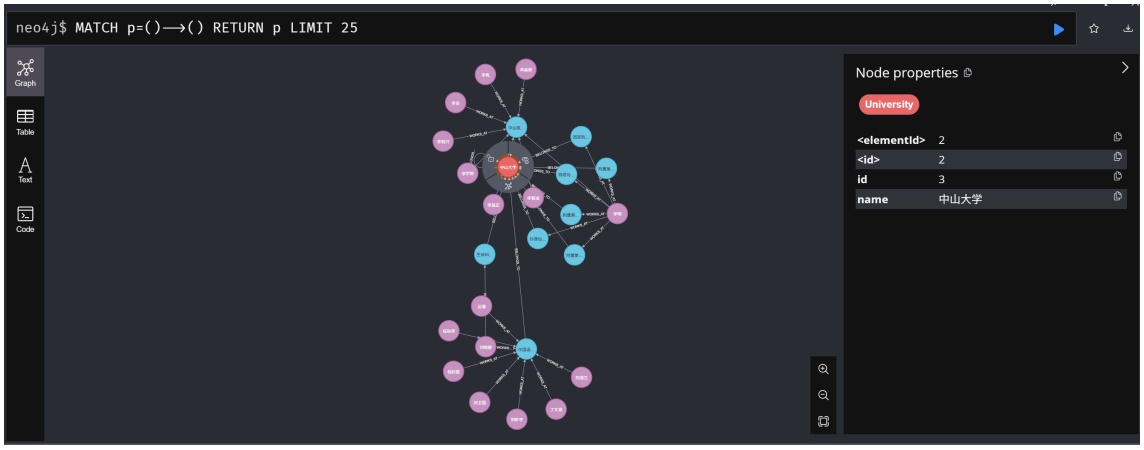


图 1.1 完整的关系图

1.8.2 学校与各学院的关系图

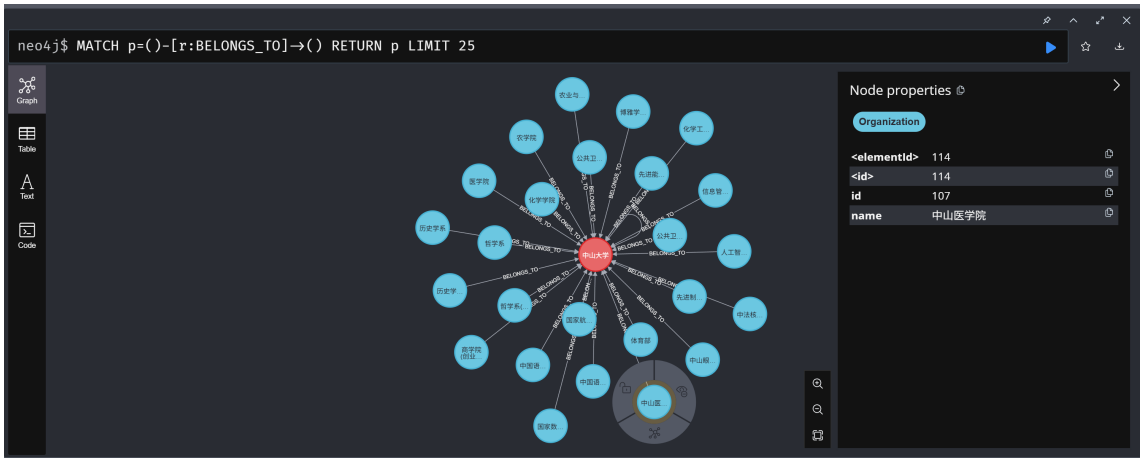


图 1.2 学校与各学院的关系图

1.8.3 各学院与老师的关系图

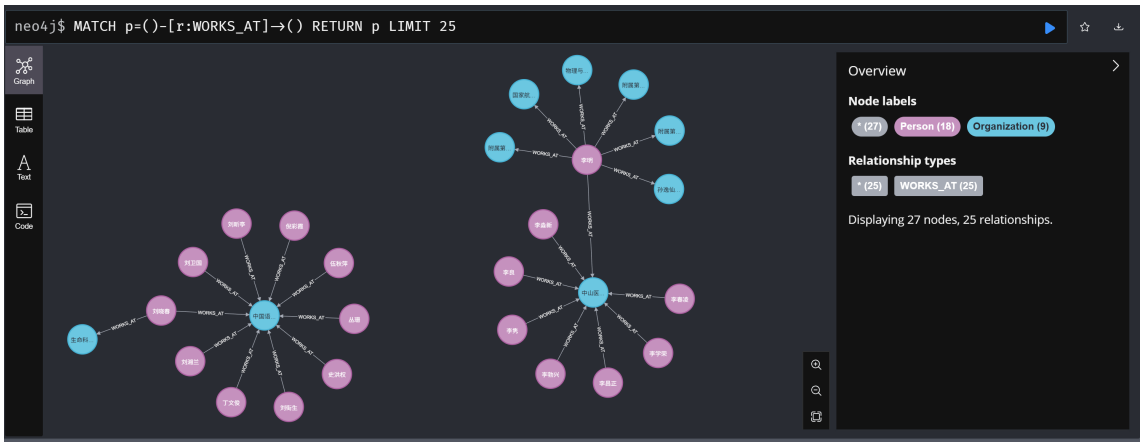


图 1.3 各学院与老师的关系图

2 总结

本次实验完成了从文本到图谱的端到端流程验证，证明了利用现有 NLP 工具与图数据库可以快速构建中小规模的知识图谱，后续可在抽取质量和图谱分析深度上继续增强。

2.1 实验总结

本次实验旨在构建中山大学教师信息的小型领域知识图谱，通过网页采集、文本清洗、命名实体识别与关系抽取，最终将提取结果导入 Neo4j 并完成可视化展示。实验验证了基于规则与现成 NLP 工具（spaCy）在结构化基础信息抽取任务中的实用性，能够较为稳定地抽取出教师姓名、工作单位与学院-学校层级关系，并以图数据库形式呈现节点与关系。

2.2 主要成果

- 成功采集并清洗了目标学院的教师简介文本，生成了用于抽取的原始语料和结构化 CSV/JSON 文件；
- 基于 spaCy 与简单规则实现了实体（Person、Organization、University）的识别，以及 WORKS_AT、BELONGS_TO 等关系的抽取；
- 使用 py2neo 将抽取结果批量导入 Neo4j，构建了可交互的知识图谱，并在报告中给出多张可视化截图以展示图谱结构与关键节点；
- 给出了核心代码片段、Cypher 查询示例与数据存储格式，具备基本复现性。

参考文献

[1] 王昌栋. 第一章绪论[Z]. 2026.

附录 A 实验代码

A.1 数据预处理代码

以下是数据预处理的 Python 实现代码：

```python
import spacy
import json
import re
import pandas as pd

def extract_information(description):
    nlp = spacy.load("zh_core_web_sm")
    doc = nlp(description)

    # Extract name
    name_match = re.search(r"\*\*(.*?)\*\*", description)
    name = name_match.group(1) if name_match else ""

    # Extract workplace and department
    workplace = ""
    department = ""
    for ent in doc.ents:
        if ent.label_ == "ORG":
            if "大学" in ent.text:
                workplace = ent.text
            else:
                department = ent.text

    return {
        "name": name,
        "workplace": workplace,
        "department": department
    }

def process_data(input_file, output_file):
```

```
    # Load data
    df = pd.read_csv(input_file)

    # Extract information from description field
    extracted_data = []
    for _, row in df.iterrows():
        description = row.get("description", "")
        if description:
            info = extract_information(description)
            extracted_data.append({
                "subject": info["name"],
                "relation": "工作于",
                "object": info["workplace"],
                "department": info["department"]
            })

    # Save extracted data to JSON
    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(extracted_data, f, ensure_ascii=False, indent=4)

if __name__ == "__main__":
    input_file = "data/preprocess_data.csv"
    output_file = "data/entities_relations.json"
    process_data(input_file, output_file)
```

A.2 关系提取代码

以下是关系提取的 Python 实现代码：

```
import csv
import json
from neo4j import GraphDatabase

def create_neo4j_data(input_file, output_file):
    nodes = []
    edges = []
    node_ids = {}
    node_counter = 1
```

```
10
11  with open(input_file, "r", encoding="utf-8") as csvfile:
12      reader = csv.DictReader(csvfile)
13      for row in reader:
14          # Add Person node
15          person_name = row["derived_teacher_name"]
16          if person_name not in node_ids:
17              node_ids[person_name] = str(node_counter)
18              nodes.append({"id": str(node_counter), "label": "Person", "
    properties": {"name": person_name}})
19              node_counter += 1
20
21          # Add Organization node
22          organization_name = row["college_name"]
23          if organization_name not in node_ids:
24              node_ids[organization_name] = str(node_counter)
25              nodes.append({"id": str(node_counter), "label": "Organization
    ", "properties": {"name": organization_name}})
26              node_counter += 1
27
28          # Add University node
29          university_name = row["school_name"]
30          if university_name not in node_ids:
31              node_ids[university_name] = str(node_counter)
32              nodes.append({"id": str(node_counter), "label": "University",
     "properties": {"name": university_name}})
33              node_counter += 1
34
35          # Add edges
36          edges.append({"source": node_ids[person_name], "target": node_ids
    [organization_name], "label": "WORKS_AT"})
37          edges.append({"source": node_ids[organization_name], "target":
    node_ids[university_name], "label": "BELONGS_TO"})
38
39  # Save to JSON file
40  with open(output_file, "w", encoding="utf-8") as jsonfile:
41      json.dump({"nodes": nodes, "edges": edges}, jsonfile, ensure_ascii=
    False, indent=4)
```

```
42
43 if __name__ == "__main__":
44     input_file = "data/preprocess_data.csv"
45     output_file = "data/neo4j_data.json"
46     create_neo4j_data(input_file, output_file)
```

A.3 Neo4j 图谱构建代码

以下是 Neo4j 图谱构建的 Python 实现代码:

```
1  from py2neo import Graph, Node, Relationship, NodeMatcher
2  import json
3
4  def import_to_neo4j(uri, user, password, json_file):
5      # Connect to the Neo4j database
6      graph = Graph(uri, auth=(user, password))
7      print("Connected to Neo4j Database")
8
9      # Load JSON data
10     with open(json_file, 'r', encoding='utf-8') as f:
11         data = json.load(f)
12
13     # Create nodes
14     for node in data['nodes']:
15         n = Node(node['label'], id=node['id'], name=node['properties']['name'
       ])
16         graph.merge(n, node['label'], 'id')
17
18     # Create relationships
19     for edge in data['edges']:
20         matcher = NodeMatcher(graph)
21         source_node = matcher.match(id=edge['source']).first()
22         target_node = matcher.match(id=edge['target']).first()
23         if source_node and target_node:
24             rel = Relationship(source_node, edge['label'], target_node)
25             graph.merge(rel)
26
27 if __name__ == "__main__":
```

```
    # Update these with your Neo4j connection details
    uri = "http://localhost:7474"
    user = "neo4j"
    password = "neo4jneo4j"

    # Path to the JSON file
    json_file = "data/neo4j_data.json"

    # Import data to Neo4j
    import_to_neo4j(uri, user, password, json_file)
```